MINIOS

Technical Documentation

Design, implementation, architecture, and testing of a command-line operating-system simulator built in Python.

Document overview

This document explains the MiniOS design and implementation based on the supplied Python source code. It describes the main components, data flow, testing checks, and key design limitations.

Contents

1. Introduction
2. Project Objectives
3. System Architecture
4. Program Structure
5. File Management
6. Process Management
7. Memory Management
8. User Management
9. Error Handling
10. Testing
11. Problems Encountered
12. Conclusion

Implementation note: The supplied MiniOS implementation is organized as one Python script. The classes described below are logical components within that file, not separate .py modules.

1. Introduction

MiniOS Alpha is a command-line operating-system simulator written in Python. It demonstrates core operating-system ideas through a text-based shell: user login, file and directory management, memory allocation, process listing, system information, and command dispatch.

The program does not boot a computer or replace a real operating system. Instead, it models selected OS behaviors in Python objects. The file manager is connected to a real folder named MiniOS_Disk, while process and memory information are maintained as simulated in-memory state.

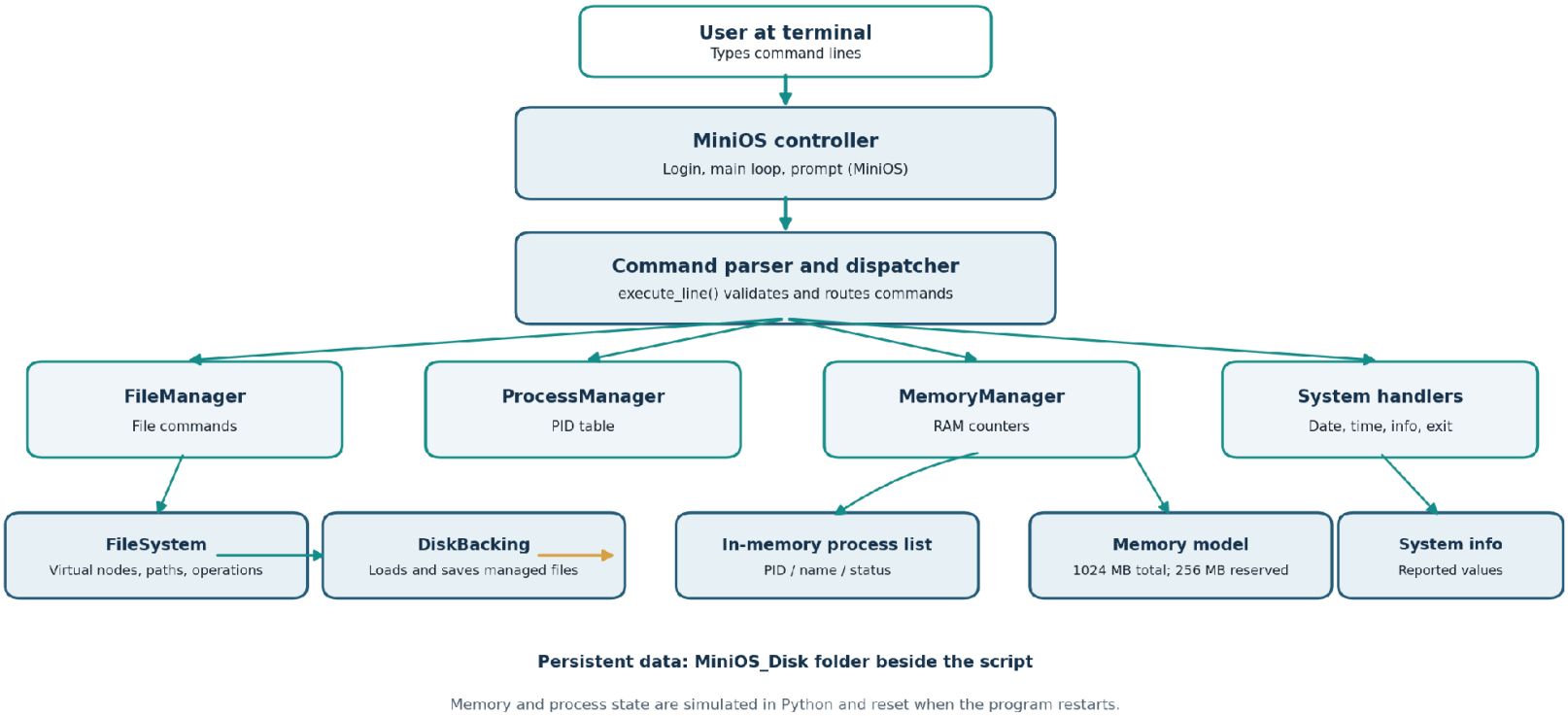
> **Scope:** MiniOS is an educational prototype. Its process entries are not real operating-system processes, and memory commands update counters rather than allocating physical RAM.

2. Project Objectives

- Build a text-based command shell that accepts user commands and displays results or errors.
- Implement a simple login flow with two built-in demonstration accounts.
- Represent files and directories as nodes in a virtual file-system tree.
- Persist managed file and directory changes to a folder on the host computer.
- Demonstrate basic process-table operations using process IDs (PIDs).
- Demonstrate memory allocation and release using a fixed simulated memory pool.
- Validate user input and prevent common invalid file-system or resource operations.
- Keep the implementation understandable by separating behaviors into classes within one script.

3. System Architecture

MiniOS follows a layered, command-driven design. The user interacts with the shell; the shell handles login and parses command lines; the dispatcher routes each command to the corresponding handler or manager. Managers return results or raise a controlled error. File operations are additionally synchronized with disk storage.



Command flow

1. The user enters a command at the MiniOS prompt.
2. MiniOS.execute_line() splits the input into a command name and arguments.
3. Built-in commands are routed through the registered command dictionary. File commands are routed to FileManager.
4. The selected manager changes state or returns formatted output. Mutating file commands trigger disk synchronization.
5. MiniOS prints the output; FSError-derived problems are shown as readable error messages.

4. Program Structure

The provided implementation is a single Python source file. Its responsibilities are divided into classes and helper functions, which provide modularity without being split into separate Python modules.

Component	Responsibility
Imports and _now()	Uses Python standard-library facilities for paths, parsing, terminal input, file operations, architecture reporting, and timestamps.
FSError and subclasses	Defines application-level errors such as NotFound, AlreadyExists, NotADirectory, InvalidName, and OutOfMemory.
Node, File, Directory	Models every virtual file-system entry. Nodes track name, owner, parent, and timestamps; directories contain child nodes.
FileSystem	Resolves paths and implements create, read, write, delete, rename, copy, move, directory listing, and tree display.
DiskBacking	Loads existing contents from MiniOS_Disk and synchronizes the virtual tree back to that folder.
FileManager	Translates file-command text and arguments into FileSystem operations and saves mutations to disk.
MemoryManager	Tracks total, reserved, allocated, used, and free simulated memory, and validates allocation requests.
ProcessManager	Maintains the process list, assigns increasing PIDs, lists processes, starts entries, and terminates eligible entries.
MiniOS	Registers system commands, handles login and the main shell loop, dispatches commands, and prints output/errors.
Entry point	The if __name__ == "__main__" block creates MiniOS and starts run().

Main command groups

Group	Commands / aliases	Handler or component
General shell	help, clear, date, time, whoami, systeminfo, exit	MiniOS
File management	dir/ls, mkdir, create, read/type/cat, delete/del, write, append, rename, cd, pwd, copy, move, tree, fhelp	FileManager
Processes	process, run, kill	ProcessManager via MiniOS handlers
Memory	memory, allocate, free	MemoryManager via MiniOS handlers

5. File Management

5.1 Data model

The file system is built from three core classes. Node holds common metadata. File adds text content and computes size from the number of characters. Directory contains a dictionary of children, mapping each child name to a File or Directory object. The root directory is represented by an empty name and displayed as /.

5.2 Path handling

FileSystem.resolve() interprets absolute paths beginning with / and relative paths beginning at the current working directory. It supports . for the current directory and .. for the parent. Backslashes are normalized to forward slashes in path helpers. When creating a node, _split() resolves the parent directory and validates the final name.

- File and directory names cannot be empty, . or .., longer than 255 characters, or contain the characters <>:"/\|?*.
- Directories cannot be read as files. A file cannot be used as a directory.
- The root directory cannot be renamed, moved, or deleted.
- A non-empty directory requires delete <path> /s to remove its contents.
- The current working directory, or a directory containing it, cannot be deleted.
- A directory cannot be copied or moved inside itself or one of its descendants.

5.3 Disk backing and persistence

DiskBacking creates the MiniOS_Disk directory beside the script unless another folder is provided to FileManager. At startup, it walks that real folder and rebuilds the virtual tree. After a mutating file command (create, mkdir, write, append, delete, rename, copy, or move), FileManager calls disk.save(), which synchronizes the real folder to match the current virtual tree.

```
Example flow: create notes.txt Hello  ->  virtual File node  ->  disk.save()  ->  MiniOS_Disk/notes.txt
```

Data safety: DiskBacking.save() removes real-folder entries that are absent from the virtual tree. Keep unrelated files out of MiniOS_Disk. The loader treats files as UTF-8 text, so the current design is not intended for binary file formats.

6. Process Management

ProcessManager simulates a process table using a Python list of dictionaries. It does not create operating-system processes. Each entry contains a process ID (pid), name, and status.

PID	Name	Initial status	Purpose
1	system_idle	RUNNING	Protected core simulator entry.
2	shell	RUNNING	Represents the MiniOS command shell and is protected.
3 onward	User-defined name	RUNNING	Added by run <name>; the next PID increases after each start.

Process operations

- process: formats the process list into terminal output.
- run <name>: checks that a name is supplied, creates a new RUNNING entry, assigns the next PID, and increments the PID counter.
- kill <PID>: converts the supplied value to an integer, refuses non-numeric values, protects PID 1 and PID 2 when they are the core entries, and removes a matching non-core entry.

```
process
run Calculator
process
kill 3
```

Lifetime: The process table is created when MiniOS starts. It is not saved to disk, so user-created process entries disappear when the program exits or restarts.

7. Memory Management

MemoryManager represents RAM using integer counters measured in megabytes (MB). The default pool contains 1,024 MB in total, of which 256 MB is reserved for the system. The user can allocate only the remainder.

Value	Default / rule	Meaning
Total	1024 MB	Total simulated memory pool.
System reserved	256 MB	Always counted as used and cannot be released by the user.
User allocated	0 MB at startup	Sum of memory requested with allocate.
Used	system + allocated	Total used amount reported by memory.
Free	total - used	Amount available for another allocation.

Allocation algorithm

For allocate <MB>, parse_amount() requires exactly one positive whole number. allocate() compares the requested amount with free memory. If the request fits, the allocated counter increases; otherwise, OutOfMemory is raised. For free <MB>, release() refuses to release more memory than the user has allocated, then decreases the counter.

```
memory
allocate 100
memory
free 50
memory
```

With a default configuration, memory begins with 256 MB used and 768 MB free. Allocating 100 MB changes those values to 356 MB used and 668 MB free. Freeing 50 MB afterward returns the state to 306 MB used and 718 MB free.

Simulation boundary: These calculations do not reserve or release actual computer RAM. The counters reset when the MiniOS process is restarted.

8. User Management

MiniOS stores two demonstration accounts in the MiniOS.USER_ACCOUNTS dictionary. The _login() method displays a login banner, requests the username through input(), and requests the password through getpass.getpass(), which normally hides typed password characters.

Username	Password in source	Result after success
admin	admin123	Sets FileManager.current_user to admin.
student	student123	Sets FileManager.current_user to student.

If the credentials match, login succeeds and the main shell loop begins. If they do not match, the program prints an error and repeats the prompt. Ctrl+C or end-of-input cancels the login flow. The owner value is assigned when new virtual nodes are created.

Security limitation: The credentials are hardcoded in the Python source, and passwords are not hashed. File ownership is metadata only: the implementation does not enforce per-user read/write permissions. These accounts are for demonstration, not protection of sensitive data.

9. Error Handling

MiniOS uses FSError as a base exception for expected application errors. Specialized subclasses let the file system and memory manager report the problem with a specific message. execute_line() catches FSError for registered commands and file commands, prints Error: followed by the exception text, and keeps the shell running.

Condition	How it is handled	Example message / behavior
Unknown command	Dispatcher fails to find the name and prints a general error.	Error: <command> is not recognized. Type help for commands.
Missing arguments	Each handler checks required argument counts with _need() or explicit checks.	Prints a Usage: instruction.
Missing file/path	resolve() raises NotFound.	The path was not found.
Duplicate name	Creation and copy operations check for an existing child.	AlreadyExists is reported.
Invalid node operation	Raises errors such as IsADirectory or NotADirectory.	Explains when a file or folder is used incorrectly.
Unsafe directory operation	Protects root/current directory and rejects non-empty deletion without /s.	InvalidOperation or DirNotEmpty.
Insufficient memory	allocate() checks against free memory.	OutOfMemory includes requested and free MB.
Invalid PID or memory amount	Numeric parsing and range checks reject invalid requests.	Readable FSError-derived messages are shown.

This design separates expected user errors from normal output and helps prevent ordinary invalid commands from terminating the shell. It is not a universal exception handler: unexpected programming errors outside the FSError hierarchy may still produce a traceback.

10. Testing

The supplied source code was functionally checked with a temporary test harness using isolated temporary folders for disk-backed operations. The checks below passed during this documentation preparation. The harness was not part of the original MiniOS source file.

Test group	Actions checked	Result
Login	An invalid credential pair is rejected; valid student credentials succeed and set the active user.	PASS
File operations	Create/read, append, overwrite, make directory, copy, rename, move, delete, and non-empty-directory guard.	PASS
Persistence	Create a file, initialize a new FileManager on the same disk folder, and read the file back.	PASS
Path / error checks	Missing path, duplicate file name, invalid name, and root-deletion protection.	PASS
Memory	Default values, allocation, release, over-capacity request, and excessive release.	PASS
Processes	Initial core entries, creating/listing/killing a user entry, invalid PID, and protecting the shell PID.	PASS
Shell dispatcher	File command execution, read output, unrecognized command, memory output, and process output.	PASS

Testing method

- Run tests against a temporary MiniOS_Disk folder to avoid changing the user's normal files.
- Check both return values and user-visible terminal messages.
- Use negative tests to confirm that invalid operations raise controlled FSError-derived exceptions.
- Reload FileManager from the same backing folder to check persistence across instances.

Limitations of this test pass: The checks are functional tests of selected code paths, not a complete certification. They do not measure performance, test concurrent access, validate binary-file handling, or establish production-grade security.

11. Problems Encountered

The source code does not include a development diary, so the points below describe implementation challenges apparent from the design and the safeguards used to address them. The group should edit this section to match the actual problems encountered during development.

11.1 Keeping disk storage and the virtual tree synchronized

A file system represented in Python can diverge from the files on disk. MiniOS addresses this by loading the MiniOS_Disk tree at startup and saving after each mutating file command. The backing layer also checks that paths remain within its configured root folder.

11.2 Validating paths and preventing invalid operations

Commands can refer to missing paths, invalid names, files where directories are expected, or directories that are not safe to remove. The implementation centralizes path resolution and validates names; it also protects the root and current working directory and requires /s for non-empty directory deletion.

11.3 Modeling memory and processes without real OS APIs

A learning simulator should show resource-management behavior without pretending to control the host operating system. MemoryManager therefore changes counters, and ProcessManager stores process metadata in a list. The main limitation is that both simulated states are temporary and reset on restart.

11.4 Producing readable error messages

If every operation returned raw Python exceptions, the terminal experience would be confusing. FSError subclasses provide domain-specific error messages, while the command loop catches these errors and lets the user try another command.

11.5 Integrating the features under a single command shell

The program uses a command registry for system commands and a separate dispatch path for file commands. This allows each feature to have its own handler while retaining one prompt and a shared output/error path.

12. Conclusion

MiniOS Alpha demonstrates how operating-system concepts can be modeled with Python classes, dictionaries, lists, path handling, exception handling, and a command loop. The file system is represented as a tree and synchronized with a real folder, while process management and memory allocation are implemented as simple state models. The shell joins these parts into a single interactive program.

Through this project, the team can learn to divide a program into components, validate inputs, maintain data structures, synchronize state with disk, handle expected errors, and test both successful and invalid operations. The project also highlights important differences between a simulator and a real operating system, particularly around process execution, physical memory, authentication security, and file permissions.

> **Future improvements:** Possible next steps include splitting the script into modules, adding automated regression tests to the project, hashing passwords, enforcing file permissions, handling binary files safely, making disk synchronization more selective, and persisting process/memory state only if required by the intended learning objectives.